

BAŞLIK: API NEDİR? TEMEL KAVRAMLAR, TARİHÇE VE API EKONOMİSİ

TÜR: Kavramsal/Giriş Seviyesi Detaylı Anlatım

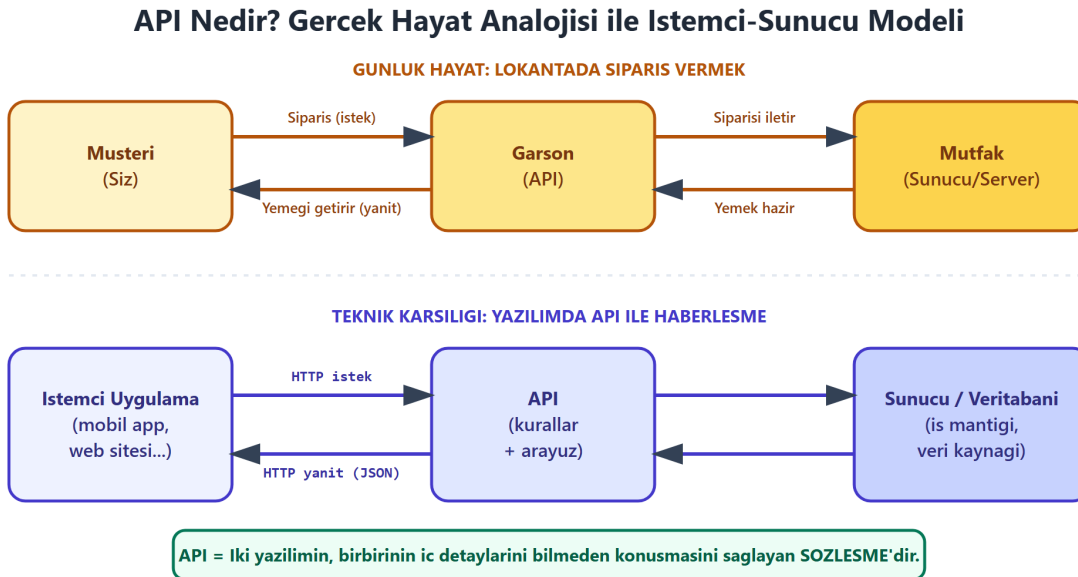
BÖLÜM 1: API (APPLICATION PROGRAMMING INTERFACE) NEDİR?

Tanım: API (Application Programming Interface / Uygulama Programlama Arayüzü), iki yazılım bileşeninin, birbirinin iç çalışma detaylarını bilmeye ihtiyaç duymadan, önceden tanımlanmış kurallar çerçevesinde veri alışverişi yapmasını ve birbirini tetiklemesini sağlayan bir SÖZLEŞMEDİR (contract).

Bu tanımın içindeki en kritik kelime "sözleşme"dir. Bir API, aslında iki taraf arasında imzalanmış bir anlaşma gibi çalışır: "Bana şu formatta bir istek gönderirsen (şu adrese, şu bilgilerle), sana şu formatta bir cevap döneceğim." Bu sözleşme sayesinde, isteği gönderen tarafın (istemci/client), cevabı üreten tarafın (sunucu/server) içeride hangi programlama dilini kullandığını, verisini nasıl sakladığını, hangi algoritmaları çalıştırdığını bilmesine gerek kalmaz. Sadece sözleşmeye uymak yeterlidir. Bu prensibe yazılım mühendisliğinde soyutlama (abstraction) ve kapsülleme (encapsulation) denir — API, karmaşık bir sistemin üzerine konan, sadeleştirilmiş ve kontrollü bir kapıdır.

1.1. Gerçek Hayattan Analoji: Restoranda Sipariş Vermek

API kavramını ilk kez duyanlar için en sık kullanılan (ve en isabetli) benzetme bir restoran sahnesidir. Bir restoranda yemek yerken doğrudan mutfağa girip aşçıyla konuşmazsınız; bunun yerine bir garson sizden siparişi alır, mutfağa iletir, mutfak yemeği hazırlar ve garson yemeği size geri getirir. Burada garson tam olarak bir API'nin rolünü oynar:



Şekil 1 — Solda gündelik hayattan garson analogisi, sağda bunun yazılımdaki teknik karşılığı: istemci uygulama, API üzerinden sunucu/veritabanına erişir.

1. Siz (müşteri) mutfağın (sunucunun) iç işleyişini bilmek zorunda değilsiniz — hangi tencereyi kullandığını, tarifin ne olduğunu bilmenize gerek yok.

2. Garson (API) size sabit bir menü (dokümantasyon) sunar: "şunları isteyebilirsiniz, bu formatta iste."
3. Mutfak (sunucu) da sizi tanımak zorunda değildir; sadece garsonun ilettiği standart siparişi işler.
4. Mutfak tariflerini değiştirse bile (iç implementasyon), garson aynı menüyü sunmaya devam ettiği sürece siz (istemci) hiçbir şey fark etmezsiniz — bu, API'lerin en kritik faydalarından biri olan "arayüz sabit kalırken iç yapının özgürce değişebilmesi" ilkesidir.

1.2. Teknik Tanım: İstemci - Sunucu Modeli

Yazılımda bu analogi birebir aynı şekilde çalışır: bir istemci uygulama (mobil uygulama, web sitesi, başka bir sunucu, hatta bir yapay zeka ajanı) bir API'ye HTTP isteği gönderir; API bu isteği alır, gerekiyorsa bir veritabanına veya iş mantığı katmanına (business logic) yönlendirir, sonucu standart bir formatta (genellikle JSON) paketleyip istemciye geri döner. Bu modülün geri kalanında en çok bu tür API'lerle — yani ağ üzerinden (network) HTTP protokolüyle çalışan Web API'leri ile — ilgileneceğiz.

BÖLÜM 2: NEDEN API'LERE İHTİYAÇ DUYARIZ?

2.1. Modülerlik ve Yeniden Kullanılabilirlik

API'ler olmasaydı, her uygulamanın ihtiyaç duyduğu her özelliği (harita gösterme, ödeme alma, SMS gönderme, hava durumu verisi çekme) sıfırdan kendi başına inşa etmesi gerekirdi. Bunun yerine, bu işlevleri uzmanlaşmış şirketler bir kere inşa eder, API olarak dışarı açar, ve binlerce farklı uygulama bu hazır yeteneği kendi ürününe entegre eder. Örneğin bir yemek sipariş uygulaması, kendi harita motorunu yazmak yerine Google Maps API'sini; kendi ödeme altyapısını kurmak yerine Stripe veya iyzico API'sini; kendi SMS altyapısını kurmak yerine Twilio API'sini kullanır.

2.2. Platformlar Arası Erişim

Modern bir ürünün genellikle birden fazla "yüzü" (istemcisi) vardır: bir web sitesi, bir iOS uygulaması, bir Android uygulaması, belki bir masaüstü uygulaması. Bu istemcilerin her biri farklı bir teknolojiyle yazılır (web: JavaScript, iOS: Swift, Android: Kotlin) ama hepsinin AYNI veriye (kullanıcı bilgisi, ürün kataloğu, sipariş geçmişi) erişmesi gerekir. Çözüm: tüm bu istemcilerin ortak bir dilde (HTTP + JSON) konuştuğu TEK bir API katmanı inşa etmek. Bu sayede iş mantığı ve veri erişimi tek bir yerde (API/sunucu tarafında) yönetilir; istemciler sadece bu API'yi çağıran ince (thin) katmanlar haline gelir.

2.3. Güvenlik ve Kontrol

API'ler, bir sistemin dışarıya açtığı YÜZEYİ (surface area) kontrol altında tutmayı sağlar. Veritabanınıza doğrudan erişim vermek yerine (ki bu son derece tehlikeli olurdu — herkes tüm veriyi silebilir, değiştirebilirdi), sadece API üzerinden izin verdiğiniz belirli işlemleri (örneğin "kendi profilini görüntüle", "sepetine ürün ekle") yapmasına izin verirsiniz. API katmanında kimlik doğrulama (authentication), yetkilendirme (authorization) ve doğrulama (validation) kontrolleri merkezi olarak uygulanır (bkz. 22.4-Kimlik_Dogrulama_ve_Guvenlik).

2.4. Ekip Ölçeklenebilirliği (Paralel Geliştirme)

Büyük şirketlerde onlarca, yüzlerce mühendis aynı anda çalışır. İyi tanımlanmış bir API sözleşmesi sayesinde, arka uç (backend) ekibi ile ön uç (frontend) ekibi BİRBİRİNİ BEKLEMEDEN paralel çalışabilir: API sözleşmesi (örn. OpenAPI/Swagger spesifikasyonu, bkz. 22.7) baştan belirlenir, frontend ekibi sahte

(mock) verilerle arayüzü inşa etmeye başlar, backend ekibi de gerçek implementasyonu yazar; sözleşmeye sadık kaldıkları sürece iki ekip aynı anda ilerler ve sonunda kusursuzca birleşirler.

BÖLÜM 3: API TÜRLERİ VE ERİŞİM MODELLERİ

3.1. Kapsamına Göre API Türleri

1. Web API'leri (bu modülün odağı): Ağ (network) üzerinden, genellikle HTTP protokolüyle erişilen API'ler. İstemci ve sunucu farklı makinelerde, hatta farklı kıtalarda olabilir (örn. Twitter/X API'si, hava durumu API'leri).
2. Kütüphane / Framework API'leri: Bir programlama dili içinde, bir kütüphanenin dışarıya açtığı fonksiyon ve sınıflar (örn. Pandas'ın `DataFrame.groupby()` metodu da teknik olarak bir API'dir — Pandas kütüphanesinin size sunduğu bir arayüzdür). Ağ üzerinden değil, aynı süreç (process) içinde çağrılır.
3. İşletim Sistemi API'leri: Bir uygulamanın, işletim sisteminin dosya sistemine, ekrana, ağ kartına erişmek için kullandığı arayüzler (örn. Windows API, POSIX sistem çağrıları).
4. Donanım API'leri: Bir yazılımın, bir donanım bileşenini (grafik kartı, kamera, sensör) kontrol etmek için kullandığı arayüzler (örn. OpenGL, DirectX).

Bu modülde "API" dediğimizde, aksi belirtilmedikçe her zaman 1. maddedeki Web API'lerini kastediyoruz — çünkü veri bilimi ve modern uygulama geliştirmede en sık karşılaşılan ve entegrasyon gerektiren tür budur.

3.2. Erişim İznine Göre API Türleri

1. Genel (Public / Open) API'ler: Herkesin (genellikle ücretsiz veya ücretli bir kayıt sonrası) kullanabildiği API'ler. Örnekler: OpenWeatherMap (hava durumu), GitHub API, Twitter/X API, PokeAPI.
2. Özel (Private / Internal) API'ler: Sadece bir şirketin kendi iç ekipleri/servisleri tarafından kullanılan, dışarıya kapalı API'ler (örn. bir e-ticaret şirketinin kendi mobil uygulamasının kullandığı, dışarıya dokümente edilmemiş iç API'ler).
3. Ortak (Partner) API'ler: Belirli, önceden anlaşılmış iş ortaklarına özel erişim verilen API'ler (örn. bir havayolu şirketinin, sadece anlaşmalı seyahat acentelerine açtığı rezervasyon API'si).

BÖLÜM 4: API EKONOMİSİ — NEDEN BUGÜN HER ŞİRKET BİR API ŞİRKETİ?

2020'lerde yazılım endüstrisinde "API Ekonomisi" (API Economy) adı verilen bir kavram öne çıktı: şirketlerin, ürünlerinin çekirdek değerini doğrudan bir API olarak paketleyip sattığı iş modelleri. Bu, geleneksel "bitmiş bir ürün satmak"tan, "bir yeteneği/altyapıyı diğer geliştiricilerin kullanımına açmak"a geçişi temsil eder.

- Stripe: Ödeme altyapısını bir API olarak sunar; şirket değeri (2024 itibarıyla onlarca milyar dolar) neredeyse tamamen bu API'nin kalitesine ve geliştirici deneyimine dayanır.
- Twilio: SMS, sesli arama, e-posta gönderme yeteneklerini API olarak satar; kod yazarak birkaç dakikada bir uygulamaya SMS doğrulama eklenebilir.
- Google Maps Platform: Harita, konum, rota hesaplama yeteneklerini API olarak sunar; milyonlarca uygulama kendi harita motorunu yazmak yerine bu API'yi kullanır.
- OpenAI / Anthropic: Büyük dil modellerinin (LLM) yeteneklerini bir API üzerinden sunar — bu repository'nin 20-Generative_AI_and_Prompt_Engineer modülünde tam olarak bu tür API'leri kullanmıştık.

Bu şirketlerin ortak noktası: ürünleri artık bir "uygulama" değil, bir "yetenek"tir ve bu yetenek API sözleşmesi üzerinden kiralanır/satılır. Bir API'nin kalitesi (tutarlılık, dokümantasyon kalitesi, güvenilirlik, performans) doğrudan o şirketin gelirini etkiler — bu yüzden modülün ilerleyen bölümlerinde göreceğimiz REST tasarım prensipleri (bkz. 22.3), güvenlik standartları (bkz. 22.4) ve dokümantasyon pratikleri (bkz. 22.7) sadece

akademik detaylar deęil, gerek iř deęeri yaratan mhendislik kararlarıdır.

SONU VE MODL YOL HARİTASI

Bu blmde bir API'nin ne olduęunu, neden var olduęunu ve iř dnyasında nasıl bir deęer yarattıęını kavramsal olarak ğrendik. Bu modln geri kalanında, sırasıyla:

1. API'lerin "tařıma dili" olan HTTP protokoln derinlemesine inceleyeceęiz (22.2).
2. İyi bir API'nin nasıl tasarlanacaęını (REST prensipleri, kaynak modelleme, sayfalama) ğreneceęiz (22.3).
3. Bir API'yi ktye kullanımdan nasıl koruyacaęımızı (kimlik doęrulama, yetkilendirme, hız sınırlama) greceęiz (22.4).
4. Python'un requests ktphanesiyle gerek API'leri nasıl tketeceęimizi, dayanıklı (resilient) bir istemci nasıl yazacaęımızı uygulamalı olarak ğreneceęiz (22.5).
5. FastAPI ile kendi production-grade REST API'mizi sıfırdan inřa edeceęiz (22.6).
6. API'lerin nasıl dokmante edilip test edileceęini greceęiz (22.7).
7. REST'in tesindeki mimarileri (GraphQL, gRPC, Webhook) karřılařtıracaęız (22.8).
8. Son olarak, bir API'nin gerek dnyada nasıl ynetildięini ve leklendięini (API Gateway, mikroservisler, izleme) inceleyeceęiz (22.9).